

Guía Paso a Paso: 1 - MovieNexus

Bienvenido al primer día de desarrollo de **MovieNexus**. Hoy sentaremos las bases de una aplicación profesional con Angular y SSR.

Paso 1: Creación del Proyecto con SSR

Ejecuta el siguiente comando:

```
bash
ng new MovieNexus --style css --routing true --standalone true --ssr true
```

Paso 2: Arquitectura Profesional (El Secreto del Éxito)

Para trabajar como en el mundo real, dividiremos nuestra app en tres pilares:

1. Carpeta core/ (Lógica Central)

Es el "cerebro" de la app. Aquí van cosas que se cargan una sola vez.

- **services/**: Para hablar con la API de TMDB.
- **interceptors/**: Para añadir seguridad a las peticiones.
- **models/**: Las "plantillas" de nuestros datos (Interfaces).

2. Carpeta shared/ (Componentes Reutilizables "Bobos")

Aquí van los componentes reutilizables. Se les llama "bobos" porque no tienen lógica de negocio; solo reciben datos y los muestran de forma bonita (ej: un botón, un spinner, una tarjeta).

- **components/layout/**: Aquí guardamos el **Header** y **Footer**, ya que definen el esqueleto de la web

3. Carpeta features/ (Componentes "Inteligentes")

Representan las **páginas** de la app (Home, Detalles, Buscador, etc). Son "inteligentes" porque ellos deciden qué datos mostrar, llaman a los servicios y orquestan a los componentes bobos de la carpeta shared.

IMPORTANTE

Regla de Oro: Si un componente se usa en muchas páginas, va a shared. Si representa una página completa o una funcionalidad única, va a features.

TIP

¿Qué es el archivo .gitkeep? Como hasta ahora estamos iniciando con el proyecto algunas carpetas que se crean, están vacías por el momento, crea este archivo .gitkeep vacío dentro de las carpetas vacías. Su función es asegurar que Git (el sistema de control de versiones) no ignore estas carpetas mientras están vacías, manteniendo nuestra arquitectura profesional intacta.

Paso 3: Sistema de Diseño (styles.css)

Usaremos **Variables CSS** para que el diseño sea "Premium" y fácil de cambiar:

```
:root {
  --bg-color: #0f172a;    /* Fondo oscuro */
  --primary: #38bdf8;     /* Azul principal */
  --accent: #f43f5e;      /* Color de destaque */
  --text-main: #f8fafc;   /* Texto principal */
  --radius: 12px;         /* Bordes redondeados */
}
```

Ejemplo de uso:

```
.mi-boton { background-color: var(--primary); border-radius: var(--radius); }
```

Paso 4: Conexión con el Mundo Exterior (API Keys)

En aplicaciones profesionales, las API Keys y URLs no se escriben directamente en los servicios. Usamos archivos de entorno:

¿Qué es una API Key?

Imagina que la base de datos de películas es un **club privado**. La **API Key** es tu **carnet de identidad** o pasaporte digital.

- **Identificación:** El servidor sabe quién pide la información.
- **Seguridad:** Evita que personas malintencionadas saturen el sistema.
- **Límites:** Como es gratuito, nos permiten un número generoso de consultas diarias.

¿De dónde salen las URLs?

- <https://api.themoviedb.org/3>: Es la "oficina central" de datos.
- <https://image.tmbd.org/t/p>: Es el servidor dedicado exclusivamente a los posters.

Configuración de Entorno

1. Ejecuta: `ng generate environments`.
2. En `src/environments/environment.development.ts` y escribe este código

```
export const environment = {
  production: false,
  baseUrl: 'https://api.themoviedb.org/3',
  apiKey: 'TU_API_KEY_AQUI',
  imgPath: 'https://image.tmbd.org/t/p'
};
```

TIP Para obtener tu llave: Regístrate en themoviedb.org, ve a tu Perfil -> Ajustes -> API y solicita una clave de tipo "Developer"

Developer Plan

No monthly fee.

Nombre de la aplicación

URL de la aplicación

Tipo de uso

Desktop Application

Resumen de la aplicación

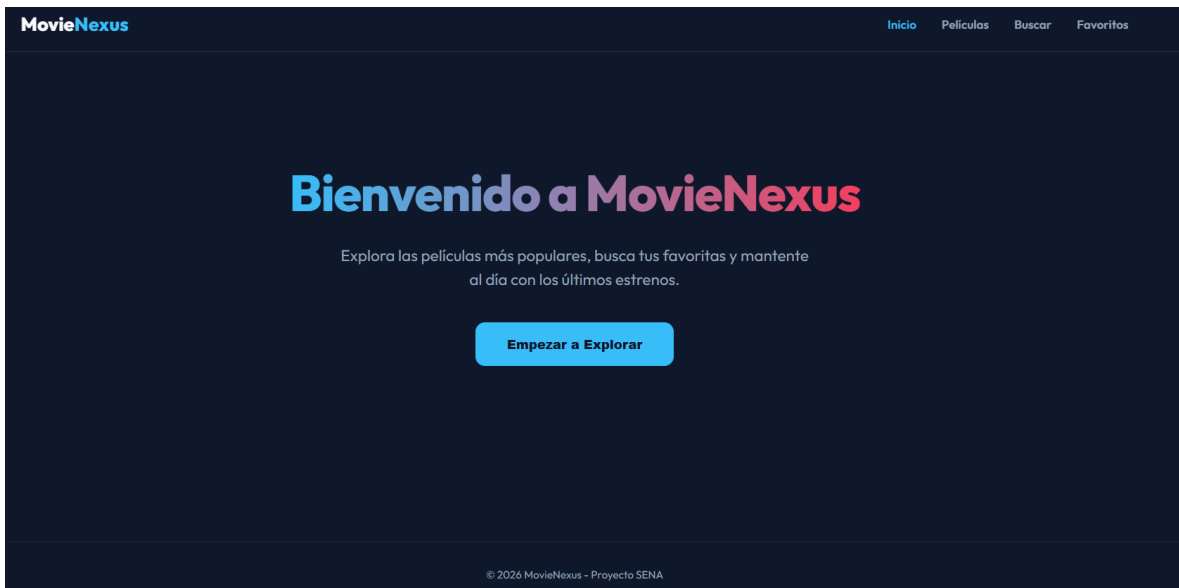
"Una aplicación web educativa construida con Angular para visualizar información de películas y series de televisión usando la API de TMDB. Proyecto de aprendizaje de integración de APIs."

Paso 5: Creación de la Primera Feature

1. Genera el componente: `ng generate component features/home`.
2. Configura la ruta en **app.routes.ts**:

```
{
  path: '',
  loadComponent: () => import('./features/home/home').then(m => m.HomeComponent)
}
```

Paso 6: Verificar su proyecto Subir a un repositorio GitHub



Subir a Git hub

1. Crea un nuevo repositorio en Github "MovieNexus" dejalo privado
2. Inicializar el repositorio local ***git init***
3. Agregar los archivos al área de preparación ***git add***.
4. Crear tu primer Commit ***git commit -m "Proyecto MovieNexus inicializado"***
5. Vincular tu proyecto local con GitHub:

git remote add origin https://github.com/TU_USUARIO/MovieNexus.git

git branch -M main

git push -u origin main

Paso 6: Publicar su proyecto en vercel y generar la URL como evidencia para classroom

Conectar GitHub con Vercel

1. Ve a vercel.com inicia sesión usando tu cuenta de **GitHub**.
2. Una vez dentro del dashboard, haz clic en el botón **"Add New..."** y luego en **"Project"**.
3. Verás una lista de tus repositorios de GitHub. Busca el de MovieNexus y haz clic en **Import**.

Configurar el Proyecto

Vercel detectará automáticamente que es un proyecto de **Angular**. Verás una sección llamada **"Build and Output Settings"**, normalmente no necesitas tocar nada ahí (Vercel ya sabe que debe usar ng build).

Desplegar

Haz clic en el botón **"Deploy"**. Vercel comenzará a compilar tu aplicación. En un par de minutos, te dará una URL ej: (<https://movie-nexus-lime.vercel.app/>) para que puedas ver tu app online.

Notas para Angular con SSR (Server-Side Rendering):

Como activaste SSR (en el Paso 1 de esta guía), Vercel lo manejará como una **Serverless Function**.

Esto significa que:

- Tu app será muy rápida para el SEO.
- Si notas que el primer despliegue tarda un poco, es normal porque está configurando el entorno de servidor